Bachelor Thesis

Tuan Anh Ta

March 2026

Freie Universität Berlin

Department of Mathematics and Computer Science

supervised by: Prof. Dr. Katharina Baum

1 Abstract

Missing data remains a significant challenge across diverse research domains. While data recovery techniques are well-established in fields such as traffic and user-rating predictions, the medical and epidemiological sectors lack standardized methodologies for addressing data gaps. Effective data reconstruction in epidemiology is critical for downstream tasks such as, identifying "patient zero" and assessing outbreak severity.

This thesis investigates the efficacy of a Variational Graph Autoencoder in reconstructing missing values within epidemiological datasets. The model's performance was benchmarked against three baseline methods: a SimpleImputer, a CustomImputer, and a Singular Value Decomposition Imputer. Evaluation was conducted using nine distinct datasets simulated via the SIR (Susceptible-Infected-Recovered) model.

The results demonstrate that the Variational Graph Autoencoder underperforms by a significant margin compared to the baseline imputation methods. This performance gap is attributed to the model's architectural limitations in capturing the temporal dimensions of disease spread, alongside the impact of significant class imbalance within the simulated datasets. These findings suggest that while graph-based models offer potential, further integration of temporal features is necessary for accurate epidemiological data recovery.[1]

[1] This page's content was stylistically refined for academic clarity using Gemini.

Contents

1 Abstract **2**

2 List of Figures/Tables/Abbreviations **5**

3 Introduction **6**

4 Background **7**
4.1 Graph . . . 7
4.2 Attribute incomplete graph . . . 7
4.3 SIR-Model . . . 7
4.4 GCN . . . 7
4.5 Data reconstruction . . . 8
4.6 Small world network . . . 8
4.7 Backpropagation . . . 8
4.8 Edge index . . . 8

5 State of the Art **9**

6 Methodology **13**
6.1 Data creation . . . 13
6.2 VGAE creation . . . 14
6.3 Training . . . 16
6.4 Testing . . . 17
6.5 Evaluation . . . 18
6.5.1 F1-Score . . . 18
6.5.2 Matthews Correlation Coefficient . . . 18
6.5.3 Transition Metric . . . 19
6.5.4 Confusion Matrix . . . 19

7 Results **20**
7.1 F1-Score and Matthews Correlation Coefficient . . . 20
7.2 Transition metric . . . 23
7.3 Confusion Matrices . . . 24

8 Discussion **27**
8.1 Performance analysis . . . 27
8.2 In the literature . . . 29
8.3 Limitations and Improvements . . . 29
8.3.1 Datacreation . . . 29
8.3.2 Hardware and Hyperparameters . . . 30
8.3.3 Evaluation . . . 30

9 Conclusion **31**

10 AI-Assistance usage **32**
10.1 Used AI-Tools . . . 32
10.2 Examples of usage . . . 32
10.3 Reflection . . . 35

11 Appendix **36**
11.1 Code Formalities . . . 36
11.2 Confusion Matrices . . . 36
11.2.1 VGAE . . . 36
11.2.2 SimpleImputer . . . 37
11.2.3 SVD . . . 38
11.2.4 CustomImputer . . . 38
11.3 Additional figures . . . 39

2 List of Figures/Tables/Abbreviations

Abbreviation	Meaning
dataset 0	dataset with 50 matrices
dataset 1	dataset with 500 matrices
dataset 2	dataset with 5000 matrices
test dataset 0	dataset with 10 matrices
test dataset 1	dataset with 100 matrices
test dataset 2	dataset with 1000 matrices
low missingness	25% of data are missing
med missingness	33% of data are missing
high missingness	50% of data are missing
S	Susceptible
I	Infected
R	Recovered
N	Total Population
VGAE	Variational Graph Autoencoder
SVD	Singular Vector Decomposition
TP, FP, TN, FN	True Positive, False Positive, True Negative, False Negative
MCC	Matthews Correlation Coefficient

Table 1: Table of abbreviations

3 Introduction

Data missingness has been a pervasive challenge for decades. For a human observer, reconstructing an incomplete matrix by sight is an intractable task; however, machine learning methods are capable of identifying latent patterns and inferring missing values from existing observations.
This challenge was famously popularized by the "Netflix Prize"—a competition by Netflix that provided a massive, incomplete dataset of user movie ratings [1]. The objective was to develop a recommender system that could accurately predict missing ratings, sparking a surge of innovation in the computer science community. Since then, numerous imputation approaches have been refined, primarily within the realms of user-rating predictions and traffic prediction [2].
Despite these advancements, progress remains comparatively limited in the medical domain, specifically within epidemiology [2]. Incomplete data represents a significant obstacle in medical research, where accurate data recovery can have profound implications. Consider a scenario involving the infection histories of thousands of patients, characterized by temporal gaps. Without a complete dataset, tasks—such as source node identification or severity assessment—become nearly impossible. Developing robust imputation methods for SIR-type data is therefore essential for recovering the true dynamics of disease transmission.

Consequently, this bachelor thesis focuses on the development and evaluation of a model designed to infer missing values within incomplete epidemiological datasets. To this end, a Variational Graph Autoencoder [3], [4] was implemented to address the following research questions:

1. How well can Variational Graph Autoencoder reconstruct missing data from an epidemiological dataset?
2. How well does it perform against other methods?
3. How does it perform when the amount of data or missingness changes?

To address these research questions, we employ a structured experimental framework. We first generate synthetic epidemiological data by simulating disease spread according to the SIR-model [5] on a Small-World Network [6]. Each simulation spans 90 days. The resulting infection histories are organized into a feature matrix, which is then subjected to random masking at three distinct levels of sparsity: 25%, 33%, and 50%. This masked data serves as the training input for the custom-built Variational Graph Autoencoder. To rigorously evaluate the VGAE's performance, its reconstruction accuracy is compared against three distinct baseline methods:

- a SimpleImputer
- a CustomImputer
- an Imputer utilizing SVD [7].

This paper will start off with the background section, which explains the foundations; followed by the current state of the art of the field. Followed by the methodology part, in which the exact procedure for the experiments is explained. We follow up with our results, its discussion and conclusion. [2]

[2] This page's content was stylistically refined for academic clarity using Gemini.

4 Background

4.1 Graph

According to [2], a graph is defined as $G = (V, E, X_v, X_e, \phi, \varphi, T, R)$. With V being the sets of nodes, E the set of edges, X_v a feature matrix for the nodes and similar X_e for the edges. T and R are representing node and edge types respectively, with $\phi : V \longrightarrow T$ and $\varphi : E \longrightarrow R$ being functions for mapping nodes and edges to their type respectively. For this work, T,R, ϕ, φ and X_e are gonna be of no importance .

4.2 Attribute incomplete graph

According to [2], an attribute-incomplete graph G is a triple (V, X, A). With V being the set of nodes, A the Adjacency Matrix and X being the feature matrix of shape $N \times D$, with N being the number of total nodes and D number of total features.

4.3 SIR-Model

The SIR-Model is an epidemiological model, used to compute the number of susceptible, infected and recovered people [5]. Following the definition of Herbert W. Hethcote in "The Mathematics of Infectious Diseases", it follows strict transitioning rules. Individuals start in the Susceptible (S) Class, in which they are vulnerable to infections. Once in contact with a disease they move to the Infected (I) Compartment. Once the Infected period is over, they move to the Recovered (R) class. The model can be viewed as a set of differential equations [5]:

$$\frac{dS}{dt} = \alpha SI/N \tag{1}$$

$$\frac{dI}{dt} = \alpha SI/N - \beta I \tag{2}$$

$$\frac{dR}{dt} = \beta I \tag{3}$$

with β being the recovery rate and α being the infection rate [5].

4.4 GCN

GCN standing for Graph Convolutional Network designed by Kipf&Welling in [8] and can be described by the following formula:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\right) \tag{4}$$

$H^{(l)}$ describes the matrix of activation in the l-th layer, with $H^{(0)} = X$. X is the feature matrix and one of two inputs for such GCN. The other is A which will be transformed $\tilde{A} = A + I$. $W^{(l)}$ is a trainable weight matrix and $D_{ii} = \sum_j \tilde{A}_{ij}$. [3]

[3] This page's content was stylistically refined for academic clarity using Gemini.

4.5 Data reconstruction

The field of data reconstruction centers around inferring missing data values from other, given data.

4.6 Small world network

Small world networks are types of graph, that are characterized by a high clustering coefficient and low average shortest distance [6].

4.7 Backpropagation

Backpropagation is a repeated procedure that adjusts the weights of connections in a network, to minimize the loss [9].

4.8 Edge index

The edge index is a sparse version of the adjacency matrix. It consists of two rows: the first one being the source and the second the target nodes [10]. [4]

[4] This page's content was stylistically refined for academic clarity using Gemini.

5 State of the Art

The primary problem addressed in this thesis is data reconstruction. Specifically, the objective is to accurately recover missing entries within a given data matrix.

The paper "Incomplete Graph Learning: A Comprehensive Survey" by Xia et al. [2], differentiates between 3 kinds of graphs:

- Attribute-missing graphs: Some nodes are missing all of their features
- Attribute-incomplete graphs: some nodes are missing some features
- Hybrid-absent graphs: a combination of the two above

The type of data structures analyzed in this thesis are attribute-incomplete graphs, wherein the unobserved node features correspond to the discrete states S, I, or R at a specific time step.
They divided the types of imputation methods into 3 categories, with two of them being relevant for us: traditional matrix completion methods and graph learning based approaches [2].

Prior to the development of traditional matrix completion techniques, researchers frequently imputed missing values using zeros or row means. To advance beyond these rudimentary approaches, Troyanskaya et al. [7] introduced novel imputation strategies for gene expression microarray data, motivated by the high sensitivity of hierarchical and k-means clustering algorithms to missing entries. Specifically, the authors proposed techniques based on Singular Value Decomposition (SVD) and weighted k-nearest neighbors (k-NN). Their findings demonstrated that these approaches significantly outperformed prior baseline methods in terms of accuracy, with k-NN emerging as the preferred technique. In their experiments, k-NN proved to be highly robust to increasing rates of missingness and exhibited low sensitivity to parameter variations. Nevertheless, the authors emphasized the need for caution when drawing biological conclusions from imputed data and advised that such entries be explicitly marked [7], reflecting a contemporary skepticism regarding the reliability of algorithmic imputation.

Subsequently, in 2008, Candès and Recht [11] established a rigorous mathematical foundation for this approach in their work, "Exact Matrix Completion via Convex Optimization." They demonstrated that given m entries of an $n \times n$ matrix sampled uniformly at random, exact recovery is guaranteed provided that m satisfies the following inequality:

$$m \geq C * n^{1.2} * r * log(n), \tag{5}$$

the original matrix can be recovered with a very high probability. Where C is a positive numerical constant and r denotes the rank of the matrix. The fundamental approach to this recovery relies on solving an optimization problem that minimizes the rank of the target matrix. While identifying a low-rank matrix consistent with the observed data enables exact recovery, direct rank minimization is an NP-hard problem, rendering it computationally intractable for practical applications. To circumvent this challenge, Candès and Recht proposed a convex relaxation, demonstrating that the matrix can be efficiently recovered by minimizing its nuclear norm instead. The authors proved that if the above-mentioned inequality is satisfied, all entries of the matrix M can be exactly recovered. However, this theoretical guarantee is limited by its reliance on two strict assumptions: the underlying matrix must be low-rank, and the observed entries must be sampled uniformly at random. [5]

[5] This page's content was stylistically refined for academic clarity using Gemini.

To illustrate this limitation, consider a matrix composed entirely of zeros, except for a concentrated region of non-zero values in one corner. If the randomly sampled entries fail to include this specific region, exact retrieval of the original matrix becomes impossible [11].

Building upon this theoretical foundation, the authors of [12] applied these principles to develop the *SoftImpute* algorithm. This approach demonstrated remarkable computational efficiency and scalability; by leveraging the nuclear norm regularization suggested in [11] , the algorithm successfully computed a low-rank approximation for a $10^6 \times 10^6$ matrix in a matter of hours.

Despite these advancements, traditional imputation methods remain constrained by stringent assumptions regarding the underlying data [2], [13]. For instance, k-nearest neighbors (k-NN) requires a sufficiently dense dataset—a condition that cannot always be guaranteed—while Bayesian estimation relies on prior knowledge of the data distribution, which is frequently unavailable in practice [13]. To mitigate this data dependency, Xia et al. [13] proposed an adapted Random Forest algorithm, termed AWVRF. Standard Random Forest models aggregate the predictions of an ensemble of decision trees via majority voting, where each node splits based on a specific dataset attribute. When an attribute is missing at a given node, previous approaches have employed surrogate attributes as a fallback [14]. However, this strategy fails when both the primary and surrogate attributes are unobserved.
To address this vulnerability, Xia et al. introduced AWVRF, which assigns dynamic voting weights to individual trees rather than treating them equally. Trees that experience significant information loss are penalized with lower voting weights compared to their more robust counterparts. Empirical evaluations demonstrated the efficacy of this approach: when compared against baselines such as mean imputation, knnImpute, and conventional Random Forest with surrogate splits (surrRF), AWVRF outperformed the alternatives in 14 out of 22 experimental scenarios. It exhibited particularly notable improvements in both computational efficiency and prediction quality relative to the standard Random Forest model. Nevertheless, as the authors note, a current limitation of this method is its restriction to binary classification tasks, as it has not yet been extended to multi-class applications [13].

A fundamental limitation of the above-mentioned methods is their inability to leverage the structural information inherent in graph data. To address this, graph learning approaches have been introduced, which are capable of utilizing both node attributes and the underlying network topology [2]. One of the pioneering applications of this paradigm [15] was developed in the domain of recommender systems. The core objective of these systems—predicting user-item ratings—can be naturally conceptualized as a matrix completion problem.
The authors of [15] formulated this task as a labeled link prediction problem on a bipartite graph, where the disjoint sets of nodes represent users and items, and the labeled edges between them denote the corresponding ratings. A key advantage of this formulation is its ability to seamlessly integrate structured external information, such as social network connections, thereby exploiting both feature representations and topological structure. The proposed architecture consists of an encoder composed of GCN layers, which learn node embeddings from user-item interactions, followed by a bilinear decoder that predicts the missing matrix entries based on the latent representations of user-item pairs. Empirical evaluations on several benchmark datasets, which incorporated external feature and graph-based information, demonstrated that their model outperformed existing methods by a significant margin. However, on larger-scale datasets, the proposed approach achieved performance comparable to other state-of-the-art algorithms[15]. [6]

[6] This page's content was stylistically refined for academic clarity using Gemini.

Monti, Bronstein, and Bresson [16] advanced this approach by integrating GCN layers [8] with a recurrent neural network and a diffusion process, enabling their architecture to outperform standard GCN models [2].

However, the majority of these graph-learning methods are tailored specifically to recommendation systems, highlighting a need for approaches applicable to general graph-structured data. To address this gap, a generalized framework known as the Variational Graph Autoencoder (VGAE) was introduced [3], [4]. The VGAE relies on an encoder-decoder architecture. It takes as input the adjacency matrix A of an undirected, unweighted graph, alongside a node feature matrix X. These inputs are processed by the encoder, which is formally defined as:

$$q(Z|X,A) = \prod_{i=1}^{N} q(z_i|X,A) \tag{6}$$

$$q(z_i|X,A) = \mathcal{N}(z_i|\mu_i, diag(\sigma_i^2)) \tag{7}$$

with $\mu = GCN(X,A)$ being a matrix of mean vectors μ_i, similar to $log(\sigma)$ [3], [4]. In simple words, the encoder tries to depict the latent representation distribution given X and A as a Normal distribution. The researcher defined the decoder as:

$$p(A|Z) = \prod_{i=1}^{N}\prod_{j=1}^{N} p(A_{ij}|z_i,z_j), \quad \text{with} \quad p(A_{ij}=1|z_i,z_j) = \sigma(z_i^\top z_j), \tag{8}$$

where A_{ij} are the elements of A and $\sigma(\cdot)$ is the logistic sigmoid function.
As a loss function they defined:

$$\mathcal{L} = E_{q(Z|X,A)}[\log p(A|Z)] - \mathrm{KL}[q(Z|X,A) \parallel p(Z)] \tag{9}$$

with KL being the Kullback-Leibler Divergence between q and p and E being the expected Value of log p. By evaluating their model on a link prediction task within a citation network, the authors demonstrated the VGAE's capacity to learn meaningful latent representations from the input data. They investigated two variants of the architecture: one utilizing node features and another relying solely on the network structure. The structure-only variant performed competitively against state-of-the-art baselines, whereas the model incorporating node features exhibited a significant increase in predictive performance [3], [4].

Subsequent research has applied this framework to an environmental domain, specifically utilizing it to predict missing air pollution measurements [17]. However, their architecture deviates slightly from the original formulation; rather than restricting the GCN layers to the encoder, they also incorporated them into the decoder. This modification was likely implemented because the task is formulated as a continuous regression problem rather than binary link prediction. Furthermore, the loss function was adapted for this specific application. [7]

[7] This page's content was stylistically refined for academic clarity using Gemini.

Instead of relying on the standard expected value formulation, the authors employed a Mean Squared Error (MSE) loss, augmented with an additional smoothness constraint [17]:

$$\mathcal{L}(X, \Theta_1, \Theta_2, \Phi) = \frac{1}{|\Omega|} \sum_{(i,j)\in\Omega} |\tilde{X}_{ij} - X_{ij}| + \beta D_{KL}(q(z|x) \parallel p(z)) + \gamma \sum_{(i,j)} \sum_{k\in\mathcal{T}(i,j)} e^{-|j-k|} (\tilde{X}_{ij} - \tilde{X}_{ik})^2 \quad (10)$$

in which γ serves as a trainable smoothness constraint. While existing graph-learning frameworks have primarily been applied to domains such as recommender systems and traffic forecasting [2], there is a notable absence of methodologies tailored to predicting unobserved patient epidemiological states within an SIR network. Addressing this specific gap in the literature is the primary objective of this thesis. [8]

[8] This page's content was stylistically refined for academic clarity using Gemini.

6 Methodology

The experiments, we conducted for answering the opposed research questions, can be divided into 5 different sections:

1. Data creation
2. VGAE model creation
3. VGAE training
4. Testing
5. Evaluation

6.1 Data creation

The generation of synthetic data forms the empirical foundation of this study. Given the focus on SIR epidemiological models, it is necessary to simulate the underlying infection dynamics. To this end, the transmission of the COVID-19 Omicron variant was selected as the target disease model and simulated across a small-world network topology [6]. These networks were generated using a consistent set of parameters: the total number of nodes was fixed at 10,000, the minimum node degree was set to 12, and the probability of random edge creation was bounded within the interval [0.001,0.1]. The function utilized for this was the *Watts_Strogatz* function from the *igraph* library.

Utilizing these specific parameter values yields a characteristic small-world network, distinguished by a high clustering coefficient and a short average path length [6]. To initiate the simulation, a single node is selected uniformly at random from the population of 10,000 to serve as the primary source of infection. The subsequent spread of the disease is dictated by the infection probability, β, and the recovery probability, γ, in accordance with standard SIR model dynamics. γ is set to 0.1, which resembles around 10 timesteps until recovery. The determination of the infection probability, β, is a more complex process, as it is inherently dependent on the basic reproduction number, R_0. This metric is formally defined as "the average number of persons infected by an infected individual introduced into a totally susceptible population" [5]. Given that the simulation models the transmission of the COVID-19 Omicron variant, R_0 is estimated to range between 9.5 and 10.0 [18]. Using these numbers as lower and upper bound, and the following formula [19], [20]:

$$R_0 = \frac{\beta}{\beta + \gamma} \frac{< k^2 > * < k >}{< k >} \tag{11}$$

β can be determined. $< k >$ is the mean degree of the network and $< k^2 >$ the second momentum of the degree distribution. Both are calculated for each network respectively during runtime. $R_0 = 9,5$ yields an infection probability of approximately $\beta = 0.38$, whereas $R_0 = 10.0$ corresponds to $\beta = 0.48$. To ensure the comprehensive transmission of the disease throughout the entire network, the higher value of β was chosen.
With these parameters established, a randomly selected source node infected its susceptible neighbors with probability β and transitioned to the recovered state with probability γ. [9]

[9] This page's content was stylistically refined for academic clarity using Gemini.

This stochastic process was applied to all infectious nodes and simulated across 90 time steps. At each iteration, the epidemiological state of S,I or R of every node was recorded, yielding a state matrix of dimensions 10,000 x 90. Consequently, each row of this matrix represents the complete infection trajectory of a single node over the 90 time steps.
Because the primary objective of the VGAE is the reconstruction of unobserved data, the subsequent step involved masking entries within these state matrices. To systematically evaluate performance, three distinct binary masks were generated for each matrix, designed to obscure 25%, 33%, or 50% of the entries within every row uniformly at random.
To investigate the efficacy of the VGAE in matrix imputation under varying volumes of training data, three distinct datasets were constructed, comprising 50, 500, and 5,000 matrices, respectively. Each matrix within these datasets was paired with the three aforementioned masks to simulate different degrees of missingness. Following an identical methodology, three supplementary datasets containing 10, 100, and 1,000 matrices were generated to serve as independent test sets, each similarly subjected to the 25%, 33%, and 50% masking ratios. The entire data generation and graph manipulation pipeline was implemented using the Python libraries igraph (v1.0.0), networkx (v3.5), and NumPy (v2.3.4).

6.2 VGAE creation

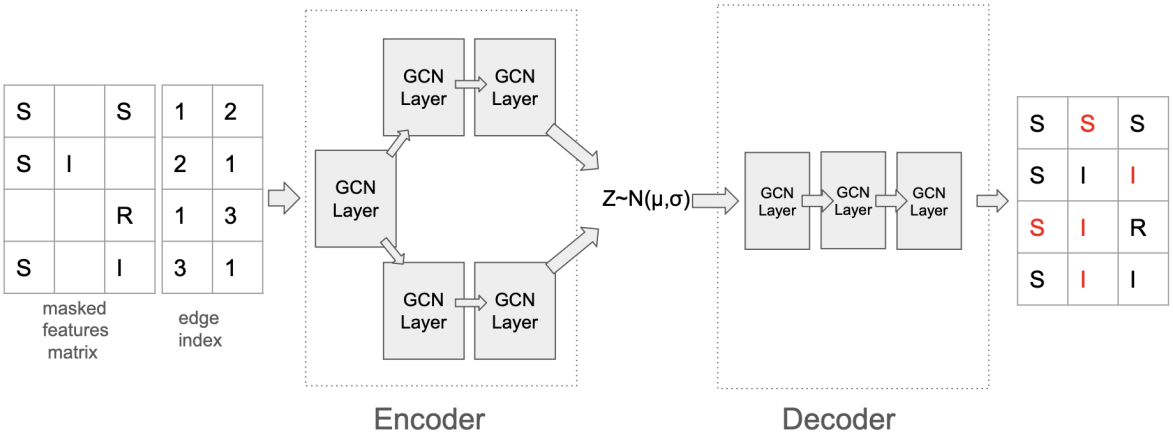

Figure 1: This figure showcases the model's architecture. It inputs a tuple of the masked feature matrix and corresponding edge index. This gets relayed through a series of GCN layers in the encoder, before being projected into the latent space Z. From there the decoder reconstructs the original matrix.

The standard VGAE comprises an encoder and a decoder [3], [4]. However, the specific VGAE implementation employed in this adapts the architecture proposed in [17], utilizing two encoders instead of one. This modification branches to learn the mean (μ) and standard deviation (σ) parameters of the underlying Gaussian latent distribution. [10]

[10] This page's content was stylistically refined for academic clarity using Gemini.

The encoder architecture begins with a shared input GCN layer, which transforms the initial input features from a dimension of 90 to an embedding dimension of 128. Following this shared layer, the architecture diverges into the distinct μ and σ branches. Each branch consists of two sequential GCN layers with both input and output dimensions set to 128, ultimately projecting the representations into a lower-dimensional latent space of dimension 10.
To reconstruct the data from this latent representation, the decoder employs an upsampling process through a corresponding series of GCN layers. The initial decoder layer maps the 10-dimensional latent vectors back to a 128-dimensional space, followed by a second GCN layer that preserves this 128-dimensional representation. An ultimate output GCN layer then projects these embeddings to a final output dimension of 270, which corresponds to the original 90 time steps multiplied by the three discrete epidemiological classes.
Throughout the network, the Rectified Linear Unit (ReLU) serves as the standard activation function for the hidden layers. The sole exception is the final layer of the σ-encoder branch, which applies a sigmoid activation function to ensure strictly positive values. To enable gradient-based optimization via backpropagation through the stochastic sampling process, μ and σ are integrated utilizing the following equation:

$$z = \mu + \sigma * e \tag{12}$$

with e being a vector which values are being randomly sampled from a normal distribution with mean 0 and variance 1. This concept is popularly known as the *reparameterization trick*[3], [4], [21]. Another fundamental aspect of the training process is the formulation of the loss function, which follows the loss function in [17]. In the proposed model, the total loss comprises three distinct components, the first of which is a categorical cross-entropy loss [22]:

$$\sum_{c=1}^{M} y_c \log(\widehat{y}_c) \tag{13}$$

M being the number of classes, y_c being the true label of class c and $\widehat{y}_c$ being the probability of predicted class.
This calculates the loss between the predictions of each class for each cell against the ground truth. The second part is the KL divergence [3], [21]:

$$\beta D_{KL}(q(z|x) \parallel p(z)) \tag{14}$$

with β being a learnable parameter, $q(z|x)$ being the approximated distribution of the latent space z given the input x and $p(z)$ being the posterior distribution [21].

The third component of the overall loss function incorporates a smoothness constraint, analogou to the formulation presented in [17]:

$$R = \gamma \sum_{(i,j)\in\mathcal{E}} \left(\sum_{k=1}^{90} X_{i,k} - \sum_{k=1}^{90} X_{j,k} \right)^2 \tag{15}$$

with γ being a learnable parameter, X the reconstructed matrix, $\mathcal{E}$ being a set of all edges. This formula sums each node's infection history subtracted by its neighbours history.
The overall objective function is formulated as the unweighted sum of these three constituent losses. [11]

[11] This page's content was stylistically refined for academic clarity using Gemini.

The model was trained for 10 epochs utilizing a batch size of 16 and a learning rate of 10^{-4}, which gets adapted using the Adam optimizer [23]. Additional hyperparameters include a KL divergence coefficient of $\beta = 0.1$ and a smoothness coefficient of $\gamma = 0.8$.
Initial empirical evaluations revealed a significant disparity in predictive performance across the three epidemiological states. While the Susceptible and Recovered classes achieved high F1-scores, the model exhibited poor predictive capability for the Infected class. Subsequent analysis of the underlying data distribution identified a severe class imbalance: Recovered states accounted for approximately 70% of the dataset, Susceptible states for 20%, and Infected states for less than 10%.
To mitigate the effects of this imbalance, a class-weighted cross-entropy loss was implemented, designed to impose penalties for misclassification based on class frequency. Following empirical tuning, the class weights were aggressively adjusted to 0.02 for the Recovered class, 0.08 for the Susceptible class, and 0.90 for the Infected class.

6.3 Training

The training pipeline initiates by applying the binary mask to the complete state matrix. The VGAE receives this partially observed matrix, alongside the corresponding edge index, as its input. The encoder processes these, to parameterize the mean μ and σ of the approximate posterior distribution in the latent space. A latent embedding is subsequently sampled from this distribution utilizing the reparameterization trick [3], [4]. The decoder then utilizes this to reconstruct the full state matrix. Finally, the reconstructed output is evaluated by the loss function, and the network weights are optimized via backpropagation.
This systematic training procedure was applied across all three datasets and their three respective types of masks, yielding a total of nine distinct experimental configurations. [12]

[12] This page's content was stylistically refined for academic clarity using Gemini.

6.4 Testing

Comparative evaluations were conducted across four distinct imputation methods: VGAE, SVD, SimpleImputer, and a CustomImputer. Independent test sets were generated following the identical procedural methodology used for the training data and corresponding masks. Specifically, the three test datasets comprise 10, 100, and 1,000 matrices, respectively, thereby maintaining a standard 80/20 train-test split relative to their training counterparts. To ensure a rigorous and equitable comparison, all methods were evaluated on identical datasets and subjected to the exact same masking configurations.

The testing pipeline for the VGAE closely mirrors its training counterpart, maintaining an identical model architecture. During this phase, the previously optimized weights are loaded into the network to leverage the learned representations. The model receives the masked state matrix, alongside the corresponding edge index. This data is processed by the encoder to map the network structure and node features into the latent space. Subsequently, the decoder samples from this, applies the reparameterization trick, and outputs a reconstruction of the complete input matrix.

In contrast to the VGAE, the SimpleImputer does not utilize the graph's edge indices; it operates solely on the masked matrix. This baseline approach imputes missing entries by replacing them with the most frequent value observed within each respective column. To implement this method, the *SimpleImputer* class from the *sklearn.impute* module was employed, with the strategy parameter explicitly configured to *'most_frequent'*.

Similar to the SimpleImputer, the CustomImputer operates exclusively on the masked matrix and does not utilize the graph's structural information. This baseline employs a deterministic, rule-based heuristic that mirrors the natural, chronological progression of the SIR disease model.
Specifically, it processes each row sequentially: it initially imputes missing entries with the Susceptible state until an Infected state is encountered. From that point forward, it transitions to imputing missing values as Infected until the first Recovered state is observed, after which all subsequent missing entries in that row are filled with the Recovered state.

The final baseline evaluated is the SVDImputer. Similar to the preceding baseline models, it operates solely on the masked state matrix without incorporating any graph topology. Because Singular Value Decomposition (SVD) requires a strictly complete matrix devoid of missing values, an initial pre-processing step is necessary. To achieve this, the previously described SimpleImputer is utilized to fill the missing entries.
Subsequently, the SVDImputer performs matrix factorization on this initialized matrix. It then reconstructs the data by retaining only the k most significant singular values and their corresponding vectors, which is a standard approach for capturing the most critical latent features while discarding noise [7]. For this specific experiment, the rank parameter was set to $k = 20$. This methodology was implemented utilizing the *TruncatedSVD* class from the sklearn.decomposition library.[13]

[13]This page's content was stylistically refined for academic clarity using Gemini.

6.5 Evaluation

Once the reconstructed matrices were obtained from all models, their imputation quality was systematically evaluated. Four distinct metrics were selected for this assessment: F1-score, Matthews Correlation Coefficient (MCC), confusion matrix, and a specialized transition metric.
Crucially, to isolate the models' true predictive capabilities, the first three metrics were computed exclusively on reconstructed matrices with the initial masks applied. In contrast, the transition metric was applied to the entire reconstructed matrix to evaluate the overall temporal consistency.

6.5.1 F1-Score

When comparing a classification prediction against its ground truth, accuracy is a natural approach for evaluating predictive quality. It is defined as the ratio of correctly predicted instances to the total number of instances in the dataset, following this formula:

$$\frac{TP + TN}{TP + TN + FP + FN}, \tag{16}$$

However, this metric can be highly misleading in the presence of severe class imbalance, as a model could achieve a high score simply by favoring the majority class. To address this, we utilized the F1-score, which provides a more balanced evaluation by calculating the harmonic mean of precision (the ratio of True Positives to all predicted positives) and recall (the ratio of True Positives to all actual positives). It follows this formula:

$$F1 = \frac{2 * Precision * Recall}{Precision + Recall} \tag{17}$$

$$precision = \frac{TP}{TP + FP} \tag{18}$$

$$recall = \frac{TP}{TP + FN} \tag{19}$$

and ranges from [0,1]. With 0 being the worst and 1 a perfect score. We used the *MulticlassF1Score* from the *torchmetrics.classification* library with *average=none*, which outputs the F1 score for every single class.

6.5.2 Matthews Correlation Coefficient

The second metric is the Matthews Correlation Coefficient, which is defined by this formula:

$$MCC = \frac{TP * TN - FP * FN}{\sqrt{(TP + FP)(TP * FN)(TN * FP)(TN * FN)}} \tag{20}$$

and ranges from -1 to 1. The MCC provides a more holistic view of the confusion matrix, yielding a score where 1 represents a perfect prediction, 0 indicates performance no better than random guessing, and -1 signifies a total disagreement between prediction and ground truth. The MCC has gained substantial scientific endorsement; specifically, Chicco and Jurman [24] demonstrated its superior reliability over both F1-score and accuracy in binary classification contexts. Gorodkin subsequently extended this metric to accommodate multi-class classification in "Comparing two K-category assignments by a K-category correlation coefficient" [25]. [14]

[14] This page's content was stylistically refined for academic clarity using Gemini.

This multi-category version of the MCC is particularly well-suited for tasks involving three classes, such as the SIR states evaluated here. The implementation used in these experiments is the *matthews_corrcoef* function from the *sklearn.metrics* library.

6.5.3 Transition Metric

A node's infection history follows a logical progression of states: a node is initially Susceptible, may eventually become Infected, and finally transitions to Recovered. To verify whether the models adhere to these biological constraints, a Transition Metric was developed to identify "illegal" transitions within each node's temporal history.
Examples of such invalid transitions include reverting from Infected to Susceptible or skipping the infection phase by moving directly from Susceptible to Recovered. If even a single fault is detected within a node's 90-step history, the entire sequence for that node is flagged as invalid. The final metric reports the fraction of valid sequences across the total population of 10,000 nodes.

6.5.4 Confusion Matrix

The final metric employed is the confusion matrix, which provides a detailed visualization of the predicted classes relative to the ground truth. Since the SIR model involves three discrete states, the result is a 3×3 matrix.
In this representation:

- The x-axis denotes the predicted classes (Susceptible, Infected, Recovered from left to right).
- The y-axis denotes the ground-truth classes (Susceptible, Infected, Recovered from top to bottom).

Consequently, all correctly predicted instances are located along the main diagonal. Any values outside of this diagonal represent misclassifications. To formalize this, consider the generic confusion matrix C underneath. An element C_{ij} represents the number of instances where the model predicted class j, but the actual ground truth was class i. For instance, the c would count the number of instances predicted as "Recovered" that were actually "Susceptible."

$$\mathrm{C} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

Normalized confusion matrices show a fraction or proportion instead of an absolute number, making them ideal for showing class imbalance. [15]

[15] This page's content was stylistically refined for academic clarity using Gemini.

7 Results

7.1 F1-Score and Matthews Correlation Coefficient

Analysis of the MCC across all methods reveals that the VGAE unexpectedly underperforms relative to its competitors. Contrary to initial assumptions, an increased volume of training data does not consistently guarantee superior reconstruction quality for the VGAE. While a positive correlation between dataset size and performance is observed for the 25% and 33% missingness configurations, this trend breaks down at the 50% missingness threshold. In this high-missingness scenario, the model trained on the smallest dataset actually achieves the highest MCC, although by a narrow margin. Individual analysis of the VGAE results across different datasets highlights further anomalies. Specifically, an increase in the percentage of missing data does not necessarily lead to a degradation in reconstruction performance for test dataset 1 and test dataset 2. For test dataset 0, no clear performance trend regarding missingness can be identified. Interestingly, this smallest model achieves its peak performance at the highest level of missingness, and even at its lowest missingness configuration, it outperforms the model tested at medium missingness.

Examination of the individual F1-scores for the VGAE reveals both unexpected anomalies and consistent trends. For the datasets containing 100 and 1,000 matrices, the variation in missingness fraction has no impact on performance.
In the smallest dataset, the F1-scores peak at the highest missingness configuration, mirroring the results seen with the MCC. However, comparing the 25% and 33% missingness configurations within this dataset shows a divergent trend: as missingness increases, the F1-score for the second class (Infected) improves, while the scores for the other classes decrease. Identifying a single optimal configuration for the VGAE is challenging, as each scenario presents specific trade-offs. For instance, an increase in training data generally improves the reconstruction quality of the Susceptible class. Conversely, the Recovered class reaches its peak performance within the test dataset 1. Most notably, the heavily underrepresented Infected class performs best in the smallest dataset under the highest missingness configuration. [16]

[16] This page's content was stylistically refined for academic clarity using Gemini.

In general, the impact of class imbalance is clearly evident across all results, as the Infected class consistently yields the lowest F1-scores not only for the VGAE but for all baseline methods as well. However, as the figure below demonstrates, the VGAE it is the only model that shows an improvement in the reconstruction quality of the Infected class as the fraction of missingness increases.

Figure 2: reconstruction F1 Score of the Infected class across different missingness configuration of the test dataset0

Additional figures in the appendix show a similar trend. Based on the results, the SimpleImputer is entirely incapable of reconstructing the Infected class, as evidenced by its F1-score of zero across every configuration. Conversely, this method excels at reconstructing the remaining classes, outperforming the VGAE in every scenario. This superiority is demonstrated not only through higher F1-scores for both the Susceptible and Recovered classes but also through its MCC.
The SVD baseline outperforms the SimpleImputer across all metrics, and consistently surpasses the VGAE in every experimental scenario. It exhibits excellent reconstruction quality for the Recovered class and maintains very high performance for the Susceptible class as well.
However, the primary weakness of the SVD approach remains the Infected class. Similar to the previous two methods, it struggles significantly to accurately impute this minority state, highlighting the persistent challenge posed by the data's class imbalance.

The only baseline that remains unaffected by class imbalance is the CustomImputer. This method consistently achieves the highest F 1-scores and MCC values across every experimental scenario, establishing it as the most effective approach for matrix value imputation. In several instances, it even reaches an almost perfect score. Unlike the other models, the CustomImputer also demonstrates robust performance on the Infected class, with a minimum observed F1-score of 0.82. [17]

[17]This page's content was stylistically refined for academic clarity using Gemini. The code for the figure's creation was also based on Gemini's suggestion.

test dataset 0	low		med		high	
	F1	MCC	F1	MCC	F1	MCC
VGAE	[0.37, 0.16, 0.53]	0.12	[0.26, 0.21, 0.48]	0.07	[0.59, 0.31, 0.64]	**0.40**
SimpleImputer	[0.75, 0.00, 0.91]	0.60	[0.75, 0.00, 0.91]	0.60	[0.75, 0.00, 0.91]	0.60
SVD	[0.80, 0.48, 0.96]	0.73	[0.79, 0.42, 0.95]	0.71	[0.78, 0.30, 0.94]	0.66
CustomImputer	[0.96, 0.87, 0.99]	**0.95**	[0.96, 0.86, 0.99]	0.93	[0.94, 0.82, 0.98]	0.91

Table 2: F1 and MCC scores of the Baselines for test dataset 0

test dataset 1	low		med		high	
	F1	MCC	F1	MCC	F1	MCC
VGAE	[0.26, 0.19, 0.77]	0.34	[0.26, 0.19, 0.77]	0.34	[0.26, 0.19, 0.77]	0.34
SimpleImputer	[0.87, 0.00, 0.80]	0.64	[0.87, 0.00, 0.80]	0.64	[0.87, 0.00, 0.80]	0.64
SVD	[0.95, 0.58, 0.92]	0.82	[0.94, 0.51, 0.91]	0.79	[0.92, 0.35, 0.88]	0.72
CustomImputer	**[0.99, 0.87, 0.98]**	**0.95**	[0.99, 0.86, 0.98]	**0.95**	[0.98, 0.82, 0.97]	0.93

Table 3: F1 and MCC scores of the Baselines for test dataset 1

test dataset 2	low		med		high	
	F1	MCC	F1	MCC	F1	MCC
VGAE	[0.68, 0.28, 0.59]	0.39	[0.68, 0.28, 0.59]	0.39	[0.68, 0.28, 0.59]	0.39
SimpleImputer	[0.73, 0.00, 0.91]	0.59	[0.73, 0.00, 0.91]	0.59	[0.73, 0.00, 0.91]	0.59
SVD	[0.78, 0.46, 0.96]	0.71	[0.77, 0.41, 0.95]	0.69	[0.76, 0.29, 0.94]	0.65
CustomImputer	[0.96, 0.87, 0.99]	0.94	[0.96, 0.86, 0.99]	0.93	[0.94, 0.82, 0.98]	0.91

Table 4: F1 and MCC scores of the Baselines for test dataset 2

7.2 Transition metric

As the results demonstrate, nearly every baseline method adheres to the state transition rules with high precision. The CustomImputer achieves a perfect score of 1.0 across every scenario, while the VGAE also maintains a perfect score in the majority of configurations. The only notable outlier is the SVD method, which shows a deviation from perfect consistency in test dataset 0 under the medium missingness configuration. [18]

	test dataset 0			test dataset 1			test dataset 2		
	low	med	high	low	med	high	low	med	high
VGAE	0.98	0.99	0.97	1.00	1.00	1.00	1.00	1.00	1.00
SimpleImputer	0.96	0.95	0.96	0.97	0.96	0.96	0.97	0.96	0.96
SVD	0.98	0.71	0.97	0.99	0.99	0.98	0.99	0.99	0.99
CustomImputer	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00

Table 5: transition metric scores of the Baselines

[18] This page's content was stylistically refined for academic clarity using Gemini.

7.3 Confusion Matrices

As each of the four methods was evaluated across nine different scenarios, only select confusion matrices are displayed in this section to maintain clarity. However, the complete set of confusion matrices is available in the appendix for further reference.

The following confusion matrix illustrates the VGAE's performance on test dataset 0 under the highest missingness configuration. According to the MCC, this represents the model's peak overall performance, and the F1-score confirms this was its most successful attempt at predicting the underrepresented Infected class.

However, an analysis of the second column reveals a significant discrepancy: the model labels Recovered instances as Infected more than three times as often as it correctly identifies Infected states. Despite this struggle with the minority class, the model remains highly confident in its classification of the other states. As shown in columns 1 and 3, misclassifications for the Susceptible and Recovered classes are notably low compared to the number of correct predictions.

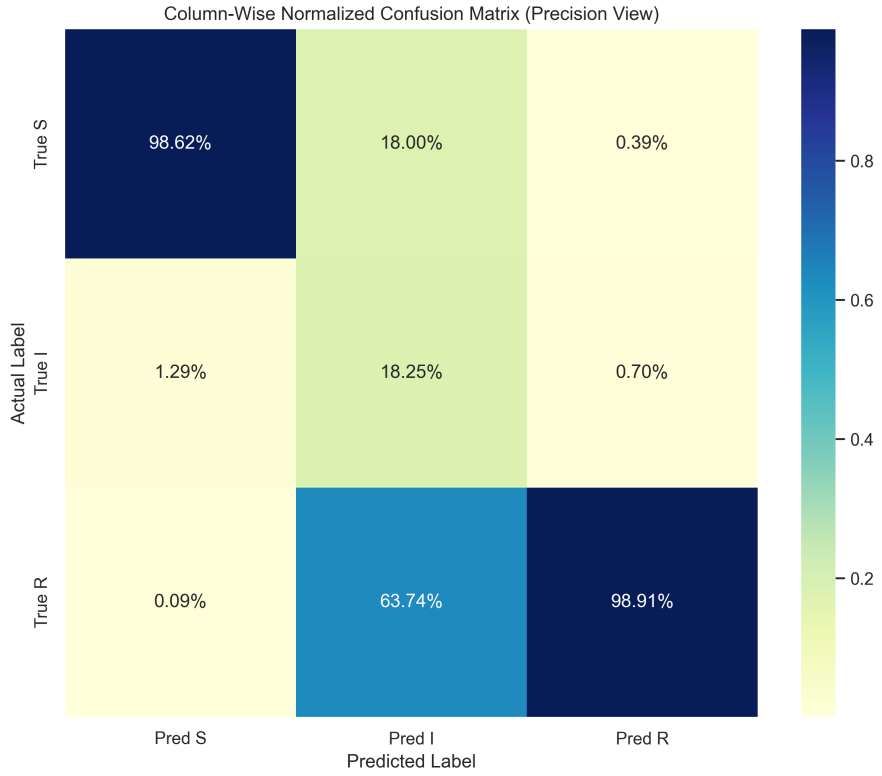

Figure 3: Confusion matrix of the VGAE in the smallest dataset on the highest missingness and normalized column-wised

[19]

[19]This page's content was stylistically refined for academic clarity using Gemini. The code for the figure's creation was also based on Gemini's suggestion.

The following confusion matrix illustrates the VGAE's performance on the smallest dataset with 33% missingness, which recorded the lowest MCC across all tests. In this configuration, the model experiences significant degradation not only in the Infected class but in the Susceptible class as well. The model demonstrates a severe bias toward the Infected state; it misclassifies Recovered and Susceptible instances as Infected nearly two and five times more often, respectively, than correctly identifying actual Infected cases. Furthermore, the reconstruction of the Susceptible class is equally problematic. The model identifies Recovered instances as Susceptible over three times more frequently than correctly predicting the Susceptible state itself.

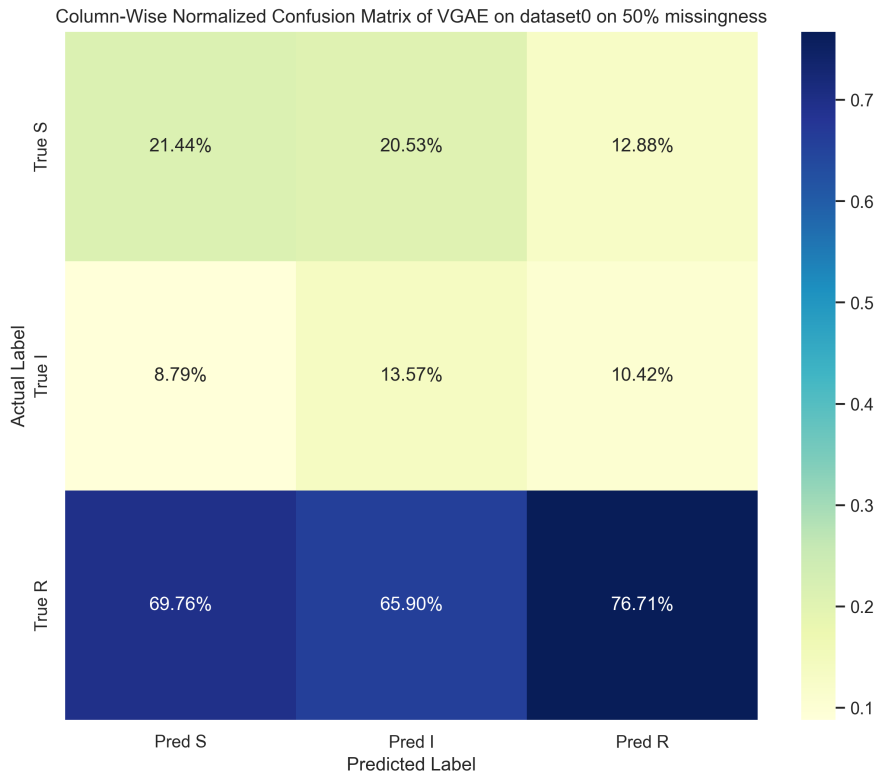

Figure 4: Confusion matrix of the VGAE in the smallest dataset on the medium missingness and normalized column-wised

[20]

[20]This page's content was stylistically refined for academic clarity using Gemini. The code for the figure's creation was also based on Gemini's suggestion.

The following confusion matrix displays the performance of the CustomImputer on the 100-matrix dataset with 25% missingness. This configuration represents one of the most successful imputations across all tests.
The reasons for this are immediately apparent. Misclassifications for both the Susceptible and Recovered classes are remarkably low. Specifically, the model correctly identifies around 90% and higher of each class, demonstrating exceptional accuracy. Furthermore, the CustomImputer achieves a perfect record in the third column: it never misclassifies a Susceptible or Infected instance as Recovered.

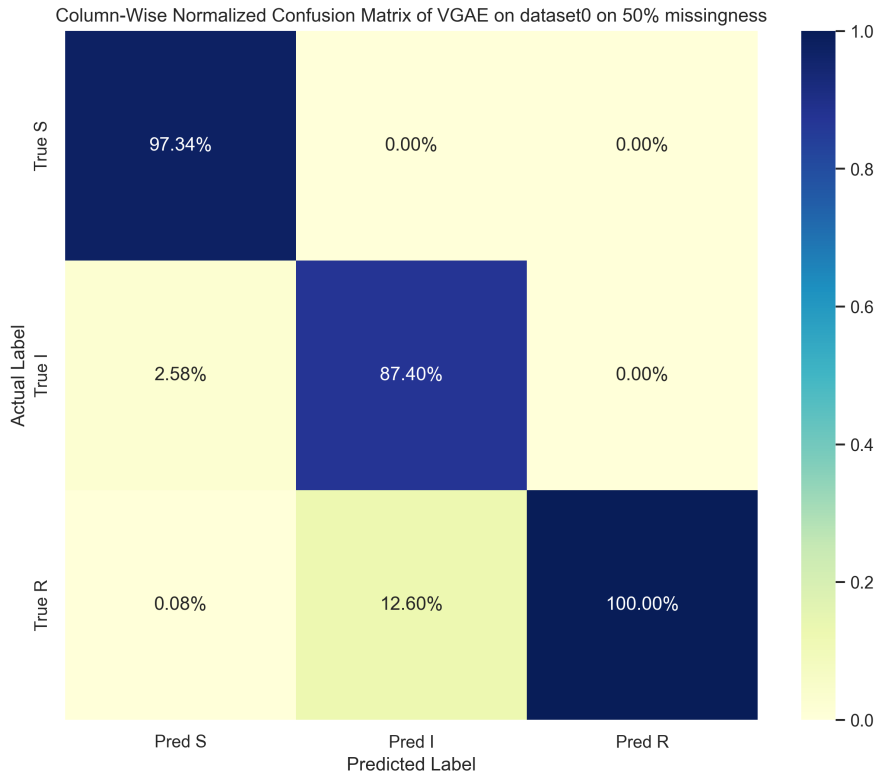

Figure 5: Confusion matrix of the VGAE in the medium dataset on the smallest missingness and normalized column-wised

[21]

[21]This page's content was stylistically refined for academic clarity using Gemini. The code for the figure's creation was also based on Gemini's suggestion.

8 Discussion

8.1 Performance analysis

The results have shown, that VGAE's arent as reliable as other baselines for the task of imputing missing SIR-data.
As previously established, the datasets contain only 10% of the Infected class. This significant imbalance likely prevents the model from observing enough distinct features to reconstruct the Infected state accurately. This limitation is why the MCC and non-averaged F1-scores were selected, as they highlight the specific effects of this disparity. The VGAE's confusion matrices further support this. In test datasets 0 and 2, a substantial number of states are incorrectly predicted as Infected instead of Recovered. This suggests that the features characterizing these two classes may be too similar for the model to distinguish effectively. In dataset 1, a similar issue occurs where many cells are classified as Infected instead of Susceptible.
These errors also indicate that the model struggles to learn temporal dependencies. While we initially assumed the GCN layers would be sufficient—following the architecture used by Do et al. [17]—research indicates that standard Graph Convolutional Networks are primarily designed to capture features in the spatial dimension.
The literature confirms that alternative layers are required to learn temporal patterns. For instance, Yu et al. [26] demonstrated in their work on traffic time-series prediction that GCN layers effectively capture spatial patterns but are limited in the temporal domain. While Recurrent Neural Networks (RNNs) are a common solution for such tasks, they successfully utilized temporal gated convolution layers to bridge this gap [26].
Additionally, the work of Li et al. proved that GCN layers function as a specialized form of Laplacian smoothing, which carries a risk of oversmoothing [27]. This phenomenon occurs when repeated graph convolutions cause the feature representations of nodes to become increasingly similar, effectively washing out the unique characteristics of individual classes [28].
This may explain the model's difficulties in distinguishing between the different epidemiological states. Specifically, the features of the minority Infected class may have "melted" into those of the more dominant Susceptible or Recovered classes, making them indistinguishable.
Furthermore, it can be inferred that once the training data exceeds a certain threshold, the fraction of missingness no longer significantly impacts the VGAE's reconstruction quality. In both test dataset 1 and Dataset 2, the F1-scores and MCC values remain consistent across all missingness configurations. This suggests that the model has developed a degree of robustness, allowing it to maintain stable performance regardless of the volume of missing data.

One could theorize that the Susceptible class would suffer from similar issues, as it is also relatively underrepresented, accounting for only 20% of the dataset. However, the results from test datasets 1 and 2 suggest otherwise. The confusion matrices for these sets reveal a much higher ratio of correct-to-false classifications compared to those observed for the Infected class. This suggests that while the model was unable to learn sufficient features from the smallest dataset, it successfully captured the characteristics of the Susceptible class as the data volume increased. In some scenarios, the model actually demonstrates higher confidence in predicting the Susceptible state than the Recovered state, despite the former being a minority class. The fact that the Susceptible state has no predecessor in the SIR model may aid in its imputation. [22]

[22] This page's content was stylistically refined for academic clarity using Gemini.

The marked discrepancy between theoretical expectations and experimental results raises a critical question: why did the VGAE underperform compared to other baselines despite having access to the edge index as an additional input? Theoretically, the VGAE should possess a distinct advantage because it learns from both the node attributes and the network topology [8], [26], indicating potential infection spreads. This spatial awareness should, in principle, provide the model with a clear edge by allowing it to leverage the states of neighboring nodes to inform its imputations. The results show otherwise though.

While the VGAE underperforms in overall metrics, it does successfully outperform the SimpleImputer in predicting the Infected class. This disparity is likely a direct consequence of the datasets class imbalance. The SimpleImputer gains a significant statistical advantage when filling in Susceptible and Recovered statuses, as Susceptible nodes naturally dominate the early stages and Recovered nodes dominate the later stages. The F1 scores confirms that the SimpleImputer thrives by leaning into these majority classes. However, the fact that the SimpleImputer fails to predict a single Infected status suggests that even at the peak of the epidemic, the number of Susceptible or Recovered nodes remained higher in a global count. This makes the SimpleImputer fundamentally unsuitable for predicting Infected statuses and, more broadly, for the general task of matrix imputation in SIR datasets.

The superior performance of the CustomImputer relative to the VGAE likely stems from how it leverages the input data. While the VGAE attempts to reconstruct the entire matrix from a latent representation, the CustomImputer is designed specifically to fill in the missing gaps within an existing structure. This specialized focus significantly reduces the margin of error, as the model can interpolate based on the known states surrounding a missing entry, allowing it to estimate exactly when a transition is likely to occur. Furthermore, the CustomImputer benefits from having the underlying transition logic embedded directly into its implementation. This prior knowledge provides a massive advantage in capturing the temporal dimension, ensuring that imputed values always remain logically consistent with the node's known history.

The SVD baseline consistently outperforms the VGAE across all test scenarios. Its strong evaluation results suggest that the 20 most significant features extracted during decomposition capture the vast majority of the data's underlying structure, leading to a highly accurate reconstruction. This performance implies that the infection history matrices possess a low rank, meaning they can be compressed and reconstructed with minimal information loss. This aligns with the theoretical framework established by Candes and Recht [11], who demonstrated that any matrix can be accurately reconstructed provided it has a sufficiently low rank. This characteristic makes SVD a robust and reliable baseline for matrix imputation tasks. The practical efficacy of SVD in high-dimensional data reconstruction is further supported by research in [7], which validates its utility in state-of-the-art experimental settings.[23]

[23]This page's content was stylistically refined for academic clarity using Gemini.

8.2 In the literature

As noted in the survey by Xia et al. regarding attribute-incomplete graph learning, modern graph-based imputation methods have demonstrated remarkable performance in various domains [2]. However, these results did not translate to this specific use case. The VGAE's underperformance can be attributed to the absence of components specifically designed to handle temporal features and extreme class imbalance. This conclusion is supported by the work of other researchers, where the integration of specialized temporal components achieved performance levels that traditional imputation methods could not match [15], [16], [17], [26]. These findings emphasize that while graph learning is a powerful tool, its success in epidemiological modeling depends heavily on its ability to capture the underlying time-dependent logic of the disease spread.

8.3 Limitations and Improvements

As the experiment was designed for a specific usecase, it comes with its own limitations which prevents it from being applicable to the general infection prediction.

8.3.1 Datacreation

The SIR model operates on the fundamental assumption that individuals who reach the Recovered status gain lifelong immunity. However, this assumption does not align with the biological reality of the specific disease chosen for this study, where reinfection may occur. Furthermore, the 90-timestamp limit for the infection simulation appears to have been insufficient for all scenarios to reach a steady state. In several instances, the simulation concluded while nodes were still in the Infected phase, meaning the disease had the potential to spread further to remaining Susceptible nodes. While these uncompleted cases accounted for less than 1% of the total population, they indicate that the chosen time span did not capture the full lifecycle of the outbreak in every simulation.

The purpose of the masking was to simulate real-world data gaps where a patient fails to report their health status. For these experiments, masks were generated uniformly by randomly hiding 25%, 33%, or 50% of each node's temporal history.
In a practical epidemiological scenario, however, data loss patterns would likely be more structured. For instance:

- Patients might report consistently until a certain timestamp and then disappear from the study entirely.
- Some individuals may never report their status at all.
- "Related" patients or those in the same geographic cluster might exhibit similar masking patterns.
- It is unlikely that every single patient in a population would have missing information; a real-world dataset would typically consist of a mix of complete and incomplete histories. [24]

[24] This page's content was stylistically refined for academic clarity using Gemini.

8.3.2 Hardware and Hyperparameters

The architecture of the GCN layers was configured with a dimensionality of 128. In contrast, the study by Do et al. [17], which served as the primary inspiration, utilized layers with a dimension of 512. This higher dimensionality likely allowed their model to capture more complex feature relationships, potentially contributing to their superior reconstruction results. Due to hardware and computational resource constraints, scaling to these larger dimensions was not feasible for our experiments. These same limitations also necessitated the exclusion of k-Nearest Neighbors, Random Forest, and Multivariate Imputation by Chained Equations as baseline methods.

Additionally, the KL-divergence and smoothness coefficient were kept identical to those used in the original paper [17]. While these hyperparameter settings provided a baseline for comparison, they were not specifically optimized for the unique characteristics of our SIR dataset.

8.3.3 Evaluation

To further evaluate the model's performance, an additional metric could have been implemented to verify whether the reconstructed infections were spatially plausible. By cross-referencing the reconstructed matrix with the adjacency matrix, this metric would determine if an infected node actually possessed an infected neighbor capable of transmitting the disease at the preceding timestamp.

For future improvements of the VGAE's F1-scores and MCC's, the integration of specialized temporal modules is essential. As the GCN layers are inherently designed to aggregate information from local neighborhoods. Two promising recommendations are [16], [26]:

- Recurrent Neural Networks
- Temporal Gated Convolutional Layers

[25]

[25] This page's content was stylistically refined for academic clarity using Gemini.

9 Conclusion

This bachelor thesis investigated the reconstruction of missing data within epidemiological datasets. A Variational Graph Autoencoder was implemented for the imputation task and evaluated against a SimpleImputer, a CustomImputer, and an SVD Imputer [7]. The empirical results indicate that the current VGAE architecture is not suited for this specific application, as it was consistently outperformed across the majority of experimental scenarios.
This underperformance can primarily be attributed to the severe class imbalance. Furthermore, scaling the volume of training data did not yield a linear improvement in reconstruction quality; while initial increases in data volume improved predictive accuracy for certain classes, performance subsequently degraded when the training size exceeded an optimal threshold. Regarding the varying degrees of data missingness, the impact on reconstruction quality became negligible once a sufficient volume of training data was provided. Below this data threshold, the effect of missingness was inconsistent, with both improvements and degradations observed as the masking ratio increased.
Across all evaluated methods, the Infected state—representing the minority class—proved the most challenging to reconstruct. While the SimpleImputer entirely failed to predict any Infected instances, the VGAE also exhibited notably poor performance, recording the lowest predictive scores for this class among the remaining models. Additionally, the VGAE demonstrated a high rate of misclassifying Susceptible and Recovered instances as Infected, suggesting an oversmoothing of node features [27]. This effect, combined with the absence of components designed to capture the temporal dimension, fundamentally constrained the model's efficacy. Because the network relied exclusively on GCN layers—which are optimized for spatial feature extraction rather than sequential dependencies—it failed to adequately model the temporal transitions inherent to the disease progression [26].

The robust performance of the SVD baseline suggests that the underlying infection data matrix possesses a low rank. This structural characteristic allows the most critical epidemiological information to be effectively compressed into a reduced number of principal components—specifically, 20 in this study—without a significant loss of information [7].
Conversely, the superior accuracy of the CustomImputer is a direct result of its algorithmic design. Embedding the SIR model's transition rules enforces logical state progressions, maximizing data utility and significantly reducing error. Furthermore, the CustomImputer operates strictly as a matrix completion tool. This targeted approach provides a distinct operational advantage over models, such as VGAE and the SVD Imputer, which attempt to reconstruct the entire data matrix from a latent representation.

In order to improve its performance, the Variational Graph Autoencoder will need to make use of some Recurrent Neural Network or Temporal Gated Convolutional Layer, which will capture the temporal dimension [26]. [26]

[26] This page's content was stylistically refined for academic clarity using Gemini.

10 AI-Assistance usage

10.1 Used AI-Tools

Solely Gemini was used for the entire process, roughly December 2025 till March 2026.

10.2 Examples of usage

Prompt: "So I have written the first draft of my entire bachelor thesis, but need your help, as it sounds not that formal, like it's a bit too informal and casual. Can you help me with that: The type of graph we are dealing with are: attribute-incomplete graphs. In our case the missing features are the statuses of either S, I or R at a certain Timestamp. "
Answer:" The underlying data structures analyzed in this thesis are attribute-incomplete graphs. Specifically, the missing node features correspond to the discrete states—S, I, or R—at a specific time step."
Usage: This answer was taken into the *State of the Art*-Section with minor modifications. A very similar approach was used for every other rewriting prombt.

Prompt: "how to make this faster:

```
def majority_class_predictor(x_input):
    for i in range(len(x_input)):
        try:
            # Find index of first 'Infected' (2)
            first_infected = np.where(x_input[i]==2)[0][0]
        except IndexError:
            first_infected = None

        try:
            # Find index of first 'Recovered' (3)
            first_recovered = np.where(x_input[i]==3)[0][0]
        except IndexError:
            first_recovered = None

        if first_recovered:
            x_input[i][:first_infected] = 1
            x_input[i][first_infected:first_recovered] = 2
            x_input[i][first_recovered:] = 3
        elif first_infected:
            x_input[i][:first_infected] = 1
            x_input[i][first_infected:] = 2
        else:
            x_input[i][...] = 1

    return x_input
```

its input is a torch.tensor"

Answer: "The current code is slow because it uses a Python loop and sequential indexing, which forces the CPU to process each row one by one. Since you are using a torch.tensor, you can leverage

vectorization to process all rows simultaneously on the GPU or CPU.
The logic you're implementing is essentially a "fill" operation based on the first occurrence of 2 (Infected) and 3 (Recovered).
The Vectorized Approach: Instead of searching for indices with np.where and looping, we can use cumulative maximums or boolean masks. This allows PyTorch to handle the logic in parallel."

```
    import torch

def fast_majority_class_predictor(x_input):
    # Ensure we are working with a torch tensor
    # 1. Create masks for where the states '2' and '3' first appear
    is_2 = (x_input == 2)
    is_3 = (x_input == 3)

    # 2. Use cummax to "spread" the state to the right
    # Once a '2' or '3' is seen, the mask remains True for the rest of the row
    has_seen_2 = is_2.cummax(dim=1)[0]
    has_seen_3 = is_3.cummax(dim=1)[0]

    # 3. Apply logic using torch.where
    # Default state is 1 (Susceptible)
    out = torch.ones_like(x_input)

    # If we've seen a 2, it becomes 2
    out = torch.where(has_seen_2, torch.tensor(2, device=x_input.device), out)

    # If we've seen a 3, it becomes 3 (this overrides the 2 because 3 comes
        later)
    out = torch.where(has_seen_3, torch.tensor(3, device=x_input.device), out)

    return out
```

Usage: This code was checked and understood and finally taken into the final codebase for the Evaluation Metrics.

Prompt: "ok how to create the graph mcc stability across increasing imbalance scenarios in python"
Answer: "To create a professional-looking MCC Stability graph in Python, the best approach is to use Seaborn combined with Matplotlib. In research papers, it is crucial to use markers and different line styles so that the readers can distinguish between models even if the paper is printed in black and white.
Python Implementation Here is the code to generate the graph. I have structured it so you can easily swap the "dummy data" for your actual results."

```python
    import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# 1. Prepare your data
# I recommend having your data in a long-format DataFrame like this:
data = {
    'Scenario': ['S1', 'S2', 'S3', 'S4', 'S5', 'S6', 'S7', 'S8', 'S9'] * 3,
    'MCC': [0.65, 0.62, 0.58, 0.50, 0.42, 0.35, 0.28, 0.20, 0.15,  # Standard VGAE
            0.66, 0.65, 0.64, 0.63, 0.61, 0.60, 0.58, 0.57, 0.55,  # Baseline B
            0.68, 0.68, 0.67, 0.67, 0.66, 0.66, 0.65, 0.65, 0.64], # Your Method
    'Baseline': ['Standard VGAE']*9 + ['GraphSMOTE']*9 + ['Proposed Method']*9
}
df = pd.DataFrame(data)

# 2. Set the aesthetic style
plt.figure(figsize=(10, 6))
sns.set_style("whitegrid", {'grid.linestyle': '--'})

# 3. Create the Line Plot
# 'hue' handles colors, 'style' handles line dashes, 'markers' adds dots/squares
ax = sns.lineplot(
    data=df,
    x="Scenario",
    y="MCC",
    hue="Baseline",
    style="Baseline",
    markers=True,
    dashes=True,
    linewidth=2.5,
    markersize=10
)

# 4. Academic Formatting
plt.title("Model Robustness (MCC) across Imbalance Scenarios", fontsize=16, pad=20)
plt.xlabel("Scenario (Increasing Imbalance Severity)", fontsize=12)
plt.ylabel("Matthews Correlation Coefficient (MCC)", fontsize=12)
plt.ylim(0, 1.0) # MCC ranges from -1 to 1, but for stability 0-1 is common
plt.legend(title="Baselines", bbox_to_anchor=(1.05, 1), loc='upper left')

# 5. Save in high resolution
plt.tight_layout()
plt.savefig('mcc_stability_graph.png', dpi=300) # For Word/Slides
plt.savefig('mcc_stability_graph.pdf')          # For LaTeX (Vector format)
```

```
plt.show()
```

Usage: The code was adjusted for displaying the F1-score of the Infected Class. This meant changing the data, labels and title. The creation was run and the resulting Figures used in this thesis.

10.3 Reflection

As this research dealt with the potential of deep learning technologies for the use in practical application, we obviously did not exclude the use but utilized it ourselves for a better result.

It was utilized mostly for debugging and style-transformation purposes. Doing these without AI would have probably yielded a similar result but at the cost of time and research.
Overall the Usage of AI has sped up the work process by a lot. It excelled at the task of rewriting given text in a formal and academic style. Though tends to make the given content longer than it needs to be. However, every sentence had to be checked and cross referenced with the original sentence in the input. This was to ensure that the meaning was not altered.
Another case, AI was used for has been the optimization of code. Especially, when it came to vectorization, a kind of technique utilized in numpy. This saves a lot of computing time. Doing this without the help of AI probably would have yielded the same result. But the research and testing of code would have taken much longer.
In terms of programming, Gemini did provide a foundational basis for the creation of figures. However, these were very simple and could have been found through a quick Google search as well. Gemini has been considered and used as a quick and efficient search engine, therefore.
For this, AI has also been utilized for researching appropriate papers. For instance, instead of asking, how some relevant concepts work, we asked for scientific papers that explain or even invented such.
AI was used for debugging as well, but only in minor cases.
In conclusion; the work of AI was very helpful. It thrives as a search engine, coder and writer. It has signaficantly sped up the work and has not disappointed in its result. The usage of AI yielded a better result, compared to not having it. However, there remains a certain lack of trust in its results. Many of its answers were double checked; we sometimes questioned its legitimacy.

11 Appendix

11.1 Code Formalities

All code was written in Python 3.13. Following is the gitHub repository containing all created code and a list of all used packages and corresponding versions:
https://github.com/schmanheq/BachelorThesis

- numpy==2.3.4
- torch==2.11.0.
- torch-geometric==2.7.0
- tqdm==4.67.1
- scikit-learn==1.8.0
- networkx==3.5
- igraph==1.0.0
- matplotlib==3.10.7
- joblib==1.5.3

11.2 Confusion Matrices

The shown Confusion Matrices in the appendix are all unnormalized.

11.2.1 VGAE

Dataset 0

$$\begin{bmatrix} 20969 & 9166 & 10258 \\ 9034 & 7780 & 7677 \\ 41859 & 50412 & 62840 \end{bmatrix} \begin{bmatrix} 18965 & 22358 & 11929 \\ 7776 & 14781 & 9650 \\ 61695 & 71783 & 71060 \end{bmatrix} \begin{bmatrix} 1 & 47690 & 586 \\ 452 & 48362 & 1056 \\ 31 & 168880 & 148390 \end{bmatrix}$$

Dataset 1

$$\begin{bmatrix} 43727 & 36995 & 35058 \\ 5902 & 7359 & 10062 \\ 19788 & 25200 & 35904 \end{bmatrix} \begin{bmatrix} 45653 & 62027 & 44956 \\ 8417 & 10279 & 12051 \\ 39620 & 29245 & 37747 \end{bmatrix} \begin{bmatrix} 34945 & 191320 & 10631 \\ 50 & 29660 & 17973 \\ 9 & 44014 & 121400 \end{bmatrix}$$

Dataset 2

$$\begin{bmatrix} 21321 & 16638 & 286 \\ 2753 & 21243 & 313 \\ 370 & 91674 & 65397 \end{bmatrix} \begin{bmatrix} 28102 & 21935 & 377 \\ 3626 & 28005 & 413 \\ 489 & 120841 & 86207 \end{bmatrix} \begin{bmatrix} 43614 & 34035 & 586 \\ 5629 & 43459 & 641 \\ 758 & 187508 & 133766 \end{bmatrix}$$

11.2.2 SimpleImputer

Dataset 0

$$\begin{bmatrix} 31972 & 0 & 8422 \\ 8867 & 0 & 15625 \\ 4378 & 0 & 150734 \end{bmatrix} \begin{bmatrix} 42028 & 0 & 11225 \\ 11514 & 0 & 20693 \\ 5624 & 0 & 198914 \end{bmatrix} \begin{bmatrix} 65188 & 0 & 17634 \\ 17677 & 0 & 32193 \\ 8585 & 0 & 308721 \end{bmatrix}$$

Dataset 1

$$\begin{bmatrix} 105344 & 0 & 10438 \\ 9625 & 0 & 13699 \\ 10313 & 0 & 70579 \end{bmatrix} \begin{bmatrix} 138899 & 0 & 13738 \\ 12720 & 0 & 18027 \\ 13643 & 0 & 92971 \end{bmatrix} \begin{bmatrix} 215468 & 0 & 21427 \\ 19650 & 0 & 28033 \\ 21052 & 0 & 144367 \end{bmatrix}$$

Dataset 2

$$\begin{bmatrix} 29419 & 0 & 8827 \\ 8836 & 0 & 15473 \\ 4469 & 0 & 152793 \end{bmatrix} \begin{bmatrix} 38796 & 0 & 11618 \\ 11659 & 0 & 20386 \\ 5901 & 0 & 201637 \end{bmatrix} \begin{bmatrix} 60234 & 0 & 18002 \\ 18118 & 0 & 31611 \\ 9174 & 0 & 312858 \end{bmatrix}$$

11.2.3 SVD

Dataset 0

$$\begin{bmatrix} 32190 & 6840.5 & 1364 \\ 6276 & 10813 & 7401 \\ 1346 & 3239 & 150526 \end{bmatrix} \begin{bmatrix} 42095 & 8622.5 & 2535.5 \\ 8521 & 12036.5 & 11650 \\ 2057 & 3882.5 & 198598.5 \end{bmatrix} \begin{bmatrix} 64766 & 11317 & 6738 \\ 14434 & 11514 & 23921 \\ 4184 & 5066 & 308056 \end{bmatrix}$$

Dataset 1

$$\begin{bmatrix} 108205 & 7540 & 38 \\ 3589 & 15901 & 3833 \\ 465 & 8001 & 72425 \end{bmatrix} \begin{bmatrix} 140974 & 11496 & 167 \\ 5725 & 18420 & 6602 \\ 1075 & 10876 & 94661 \end{bmatrix} \begin{bmatrix} 215599 & 19443 & 1853 \\ 12201 & 17868 & 17614 \\ 4338 & 16055 & 145025 \end{bmatrix}$$

Dataset 2

$$\begin{bmatrix} 29544 & 6903 & 1798 \\ 6139 & 10302 & 7867 \\ 1456 & 3194 & 152791 \end{bmatrix} \begin{bmatrix} 38760 & 8546 & 3108 \\ 8538 & 11443 & 12064 \\ 2381 & 3796 & 201360 \end{bmatrix} \begin{bmatrix} 59808 & 10947 & 7481 \\ 14811 & 10962 & 23955 \\ 5062 & 4705 & 312264 \end{bmatrix}$$

11.2.4 CustomImputer

Dataset 0

$$\begin{bmatrix} 40395 & 0 & 0 \\ 3118 & 21374 & 0 \\ 87 & 3090 & 151935 \end{bmatrix} \begin{bmatrix} 53253 & 0 & 0 \\ 4474 & 27733 & 0 \\ 207 & 4471 & 199859 \end{bmatrix} \begin{bmatrix} 82822 & 11317 & 0 \\ 8865 & 41004 & 0 \\ 816 & 8859 & 307630 \end{bmatrix}$$

Dataset 1

$$\begin{bmatrix} 115780 & 0 & 0 \\ 3069 & 20255 & 0 \\ 91 & 2920 & 77880 \end{bmatrix} \begin{bmatrix} 152637 & 0 & 0 \\ 4428 & 26319 & 0 \\ 198 & 4238 & 102177 \end{bmatrix} \begin{bmatrix} 236895 & 0 & 0 \\ 8839 & 38845 & 0 \\ 826 & 8405 & 156188 \end{bmatrix}$$

Dataset 2

$$\begin{bmatrix} 38247 & 0 & 0 \\ 3080 & 21229 & 0 \\ 93 & 3075 & 154275 \end{bmatrix} \begin{bmatrix} 50415 & 0 & 0 \\ 4457 & 27588 & 0 \\ 198 & 4459 & 202880 \end{bmatrix} \begin{bmatrix} 78237 & 0 & 0 \\ 8902 & 40826 & 0 \\ 836 & 8896 & 312300 \end{bmatrix}$$

11.3 Additional figures

Figure 6: reconstruction F1 Score of the Infected class across different missingness configuration of the test dataset1

[27]

[27] The code for the figure's creation was based on Gemini's suggestion.

Figure 7: reconstruction F1 Score of the Infected class across different missingness configuration of the test dataset2

[28]

[28]The code for the figure's creation was based on Gemini's suggestion.

References

[1] J. Bennett, C. Elkan, B. Liu, P. Smyth, and D. Tikk, "KDD Cup and workshop 2007," *SIGKDD Explor. Newsl.*, vol. 9, no. 2, pp. 51–52, Dec. 2007, ISSN: 1931-0145. DOI: 10.1145/1345448.1345459. [Online]. Available: https://doi.org/10.1145/1345448.1345459.

[2] R. Xia et al., "Incomplete graph learning: A comprehensive survey," *Neural Networks*, vol. 190, p. 107 682, 2025, ISSN: 0893-6080. DOI: https://doi.org/10.1016/j.neunet.2025.107682. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0893608025005623.

[3] T. N. Kipf and M. Welling, "Variational graph auto-encoders," *Computing Research Repository*, vol. abs/1611.07308, 2016. arXiv: 1611.07308. [Online]. Available: https://arxiv.org/abs/1611.07308.

[4] P. K. Diederik and M. Welling, "An introduction to variational autoencoders," *Foundations and Trends in Machine Learning*, vol. 12, no. 4, pp. 307–392, Nov. 2019, ISSN: 1935-8237. DOI: 10.1561/2200000056. eprint: https://www.emerald.com/ftmal/article-pdf/12/4/307/11160827/2200000056en.pdf. [Online]. Available: https://doi.org/10.1561/2200000056.

[5] H. W. Hethcote, "The mathematics of infectious diseases," *SIAM Review*, vol. 42, no. 4, pp. 599–653, 2000. DOI: 10.1137/S0036144500371907. eprint: https://doi.org/10.1137/S0036144500371907. [Online]. Available: https://doi.org/10.1137/S0036144500371907.

[6] D. J. Watts and S. H. Strogatz, "Collective dynamics of 'small-world' networks," *Nature*, vol. 393, no. 6684, pp. 440–442, 1998. DOI: 10.1038/30918.

[7] O. Troyanskaya et al., "Missing value estimation methods for dna microarrays," *Bioinformatics*, vol. 17, no. 6, pp. 520–525, Jun. 2001, ISSN: 1367-4803. DOI: 10.1093/bioinformatics/17.6.520. eprint: https://academic.oup.com/bioinformatics/article-pdf/17/6/520/48837104/bioinformatics_17_6_520.pdf. [Online]. Available: https://doi.org/10.1093/bioinformatics/17.6.520.

[8] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks.," in *ICLR (Poster)*, OpenReview.net, 2017. [Online]. Available: https://openreview.net/forum?id=SJU4ayYgl.

[9] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *nature*, vol. 323, no. 6088, pp. 533–536, 1986. [Online]. Available: https://doi.org/10.1038/323533a0.

[10] M. Fey and J. E. Lenssen, "Fast graph representation learning with pytorch geometric," *CoRR*, vol. abs/1903.02428, 2019. arXiv: 1903.02428. [Online]. Available: http://arxiv.org/abs/1903.02428.

[11] E. Candès and B. Recht, "Exact matrix completion via convex optimization," *Commun. ACM*, vol. 55, no. 6, pp. 111–119, Jun. 2012, ISSN: 0001-0782. DOI: 10.1145/2184319.2184343. [Online]. Available: https://doi.org/10.1145/2184319.2184343.

[12] R. Mazumder, T. Hastie, and R. Tibshirani, "Spectral regularization algorithms for learning large incomplete matrices," *Journal of Machine Learning Research*, vol. 11, no. 80, pp. 2287–2322, 2010. [Online]. Available: http://jmlr.org/papers/v11/mazumder10a.html.

[13] J. Xia et al., "Adjusted weight voting algorithm for random forests in handling missing values," *Pattern Recognition*, vol. 69, pp. 52–60, 2017, ISSN: 0031-3203. DOI: https://doi.org/10.1016/j.patcog.2017.04.005. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0031320317301504.

[14] A. Hapfelmeier, T. Hothorn, K. Ulm, and C. Strobl, "A new variable importance measure for random forests with missing data," *Statistics and Computing*, vol. 24, no. 1, pp. 21–34, Jan. 2014, ISSN: 0960-3174. DOI: 10.1007/s11222-012-9349-1. [Online]. Available: https://doi.org/10.1007/s11222-012-9349-1.

[15] R. van den Berg, T. N. Kipf, and M. Welling, "Graph convolutional matrix completion," *Computing Research Repository*, vol. abs/1706.02263, 2017. arXiv: 1706.02263. [Online]. Available: http://arxiv.org/abs/1706.02263.

[16] F. Monti, M. M. Bronstein, and X. Bresson, "Geometric matrix completion with recurrent multi-graph neural networks," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17, Long Beach, California, USA: Curran Associates Inc., 2017, pp. 3700–3710, ISBN: 9781510860964. DOI: 10.5555/3294996.3295127.

[17] T. H. Do et al., "Matrix completion with variational graph autoencoders: Application in hyperlocal air quality inference," in *ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019, pp. 7535–7539. DOI: 10.1109/ICASSP.2019.8683787.

[18] Y. Liu and J. Rocklöv, "The effective reproductive number of the omicron variant of sars-cov-2 is several times relative to delta," *Journal of Travel Medicine*, vol. 29, no. 3, taac037, Mar. 2022, ISSN: 1708-8305. DOI: 10.1093/jtm/taac037. eprint: https://academic.oup.com/jtm/article-pdf/29/3/taac037/43899445/taac037.pdf. [Online]. Available: https://doi.org/10.1093/jtm/taac037.

[19] Wikipedia contributors, *Basic reproduction number — Wikipedia, the free encyclopedia*, [Online; accessed 21-February-2026], 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Basic_reproduction_number&oldid=1339617236.

[20] M. E. J. Newman, "Spread of epidemic disease on networks," *Phys. Rev. E*, vol. 66, p. 016 128, 1 Jul. 2002. DOI: 10.1103/PhysRevE.66.016128. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevE.66.016128.

[21] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," in *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Conference Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2014. [Online]. Available: http://arxiv.org/abs/1312.6114.

[22] C. M. Bishop, *Pattern recognition and machine learning, 5th Edition* (Information science and statistics). Springer, 2007, ISBN: 9780387310732. [Online]. Available: https://www.worldcat.org/oclc/71008143.

[23] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2015. [Online]. Available: http://arxiv.org/abs/1412.6980.

[24] D. Chicco and G. Jurman, "The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation," *BMC Genomics*, vol. 21, no. 1, p. 6, Jan. 2020, ISSN: 1471-2164. DOI: 10.1186/s12864-019-6413-7. [Online]. Available: https://doi.org/10.1186/s12864-019-6413-7.

[25] J. Gorodkin, "Comparing two k-category assignment by a k-category correlation coefficient," *Computational biology and chemistry*, vol. 28, pp. 367–74, Dec. 2004. DOI: 10.1016/j.compbiolchem.2004.09.006.

[26] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," in *Proceedings of the 27th International Joint Conference on Artificial Intelligence*, ser. IJCAI'18, Stockholm, Sweden: AAAI Press, 2018, pp. 3634–3640, ISBN: 9780999241127. DOI: 10.5555/3304222.3304273.

[27] Q. Li, Z. Han, and X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence*, ser. AAAI'18/IAAI'18/EAAI'18, New Orleans, Louisiana, USA: AAAI Press, 2018, ISBN: 978-1-57735-800-8. DOI: 10.5555/3504035.3504468.

[28] C. Yang, R. Wang, S. Yao, S. Liu, and T. F. Abdelzaher, "Revisiting "over-smoothing" in deep gcns," *CoRR*, vol. abs/2003.13663, 2020. arXiv: 2003.13663. [Online]. Available: https://arxiv.org/abs/2003.13663.